

Virtual Cookbook Platform

Personal Digital Cookbook

From Scattered Recipes to One Kitchen

Team: Nolan Griffith, Alex Valsamis

Course: ECE-366 Software Engineering & Large Systems Design: Spring 2026

Introduction

We are building a full-stack virtual cookbook platform designed to centralize recipe management, discovery, and social sharing into a single, intuitive web application. Users create personal digital cookbooks, organize recipes into collections, and explore what others in the community are cooking.

Today, home cooks store recipes across bookmarked websites, screenshot folders, handwritten notes, and messaging threads. Finding the right recipe is often harder than making the dish. Recipes disappear behind broken links; family recipes live in notebooks that are never digitized. Our platform serves as a single source of truth: users maintain a personal cookbook, import and adapt recipes from other users, and organize everything into named collections. A community browsing layer enables public discovery. The platform is scalable, modular, and adaptable, with an emphasis on clarity, usability, and reliability.

Problem Statement

Home cooks usually rely on browser bookmarks, camera-roll screenshots, and group-chat links that create friction and limit discoverability. For communities of food enthusiasts, there is no easy way to browse what peers are cooking, copy and personalize someone else's recipe, or maintain attribution when a recipe is adapted. Existing platforms either focus on professional publishing or are locked behind paywalls and algorithmic feeds. This project introduces a centralized, interactive cookbook platform with personal organization tools, community discovery features, and structured recipe data management.

System Overview

The platform is a full-stack web application: a responsive React frontend, a secure Spring Boot backend API, and a PostgreSQL database. It supports user authentication, structured recipe data management, social features, and cloud deployment with clean separation of concerns.

Layer Technology & Details	
Frontend	React with TypeScript. Component-based UI, React Router, Axios, responsive design.
Backend	Spring Boot (Java). RESTful API, service-layer logic, input validation, Spring Security.
Database	PostgreSQL. Relational schema, JPA entities, transaction-safe ops, indexed queries.
Auth	JWT-based authentication, bcrypt hashing, configurable token expiry.
Infra	Docker/Docker Compose, cloud deploy (Render/Railway), GitHub Actions CI, env secrets.

Source code hosted in a **GitHub Organization** with branch protection and PR-based code review. We also document past issues in the Projects Section to show our progress at certain intervals throughout the demoing and debugging.

Project Management: We utilized an Iterative Sprint model focused on Milestone-Based Synchronous Development. Due to the project's high level of collaboration, we prioritized real-time communication outside of GitHub for rapid decision-making. Documentation of tasks was performed in comprehensive batches corresponding to major system milestones: the Backend core, the Frontend integration, and the Final Deployment/Polish phase. This approach allowed us to maintain high development velocity while ensuring that our GitHub project board accurately reflects the resolution of cross-stack dependencies during our final 'Integration & Polish' phase.

Core Features

User Accounts and Authentication

Secure registration with email validation and unique username. JWT-based login. A user can browse, save, create recipes, organize collections. Profile management includes display name, bio, email, edit profile, change password, and account deletion.

Personal Cookbook & Recipe Management (CRUD)

Each user has a personal cookbook displaying all authored or saved recipes. Users create recipes via a structured form: title, description, ingredients (with quantities/units), step-by-step instructions, prep/cook time, servings, category tags (Italian, Vegan, Dessert, etc.), and image options. Users can edit or delete their own recipes. Each recipe has a detail page with formatted ingredients with quantities, numbered steps, timing info, and author attribution.

Collections and Organization

Users create named collections (e.g., “Weeknight Dinners,” “Holiday Baking”) to organize recipes. A recipe can belong to multiple collections. Collections can be reordered and renamed, providing folder-based navigation for large libraries.

Community Browsing, Search, and Save/Copy

A public feed displays recipes marked as public. Keyword search across titles, descriptions, and tags with dynamic filters on category, prep time, and dietary tags. Users can save any public recipe to their cookbook with one click: creating an editable copy with attribution to the previous author via a “lineage.” A “Stash” section aggregates all copied recipes.

User Privacy and Settings

Users have their own personal cookbook where they can manage recipes in My Kitchen and Browse. Recipes can be labeled as private, so only the owner can view them, or public, so they can appear in Browse, saved recipes, collections, and the community feed for other users. Users can also update their account settings, including changing their password securely through email recovery on the login page or by entering their current password on the profile page. In addition, users can edit their bio and upload or change their profile picture.

Project Scope and Effort Estimates

Feature Key Tasks Effort Target			
User Accounts	Registration, login, JWT auth, profile, edit, password, deletion	2 wks	Mar 9
DB Schema	Schema (users, recipes, ingredients, steps, collections, saved), JDBC, seed data.	2 wks	Mar 9
Backend API	REST CRUD endpoints, validation, error handling, business logic (collections, save/copy).	3 wks	Mar 9

Dashboard UI	React pages: cookbook (grid+list), recipe detail/form, collections, feed, profile	2 wks	Apr 20
Integration	Axios API calls, protected routes, token storage, loading/error states.	2 wks	Apr 20
Search & Filters	Keyword search via string and tags	1 wk	Apr 20
Testing & QA	Exceeded targets by reaching $\geq 75\%$ line coverage across both Backend and Frontend. Utilized JaCoCo for Java and Vitest/V8 for React to verify integrity	2 weeks	May 11
Deploy & CI	Dockerfile, Docker Compose, GitHub Actions, Cloud deploy	1.5 wks	May 11

Risks and Mitigation

- **Scope management:** While image uploads were initially considered a stretch goal, we successfully implemented local Base64-encoded image processing to enhance the visual appeal of the prototype without relying on external cloud storage (S3).
- **Frontend-backend integration:** API contracts defined early in Phase 1. Frontend uses stub data before full integration.
- **Authentication complexity:** Core CRUD first; auth layered in afterward to avoid blocking the Mar 9 demo.
- **Time constraints:** Modularized tasks for parallel development. Sprints re-scoped weekly based on progress

Roles and Responsibilities

Member Role Primary Responsibilities		
Alex Valsamis	Frontend Lead	React (Vite/TS), UI architecture, routing, components, Axios, responsive design, cloud deployment, system integration
Nolan Griffith	Backend Lead	Spring Boot controllers, service layer, REST API, business logic, data workflows, Docker, testing, PostgreSQL schema, JDBC entities, seed data, JWT auth, Spring Security, bcrypt

All team members collaborate on system design, testing, and code reviews via GitHub pull requests.

Milestones

Date	Milestone	Deliverables	Tests
Feb 9	Proposal & Pitch	This document; 5-min class presentation. Architecture finalized.	—
Mar 9	DB & Backend	GitHub org, DB schema, Docker Postgres, Spring Boot $\geq 50\%$ features. Tagged release. Demo via Postman.	—
Apr 20	Local Demo	Full local stack via docker-compose up. $\geq 75\%$	$\geq 25\%$

		features: cookbook, CRUD, collections, feed, search, save/copy.	
May 5	Final	Achieved full local stack deployment via Docker Compose. Implemented advanced navigation (image-based routing), reliable account deletion with foreign-key cascading, and robust error handling for auth workflows	$\geq 50\%$

Conclusion

This project delivers a clean, scalable virtual cookbook platform that consolidates fragmented recipe management into a single, reliable application. It solves a real friction point: home cooks losing track of recipes scattered across bookmarks, screenshots, and messaging threads. By emphasizing modular design, structured data management, community features, and deployment best practices, the system is both academically rigorous and practically extensible beyond the course.